

Сборка кодинг-агента

Семь ступеней конфигурации на одном сквозном кейсе

04

«У вас репозиторий и агент, который в нём работает — какой блок конфигурации добавить первым, и как понять, что следующий добавлять ещё рано?»

ПУСТАЯ ДИРЕКТОРИЯ → СЕМЬ ДОБАВЛЕНИЙ, ПОКА БЕЗ ИМЕН



Занятие начинается не с конфигурации агента, а с требований задачи

ВИДНО В КОДЕ

НИГДЕ НЕ БУДЕТ ВИДНО

«Заказчик просит лендинг с формой заявки на демо-урок. Одна страница, два поля, кнопка. Назовите вслух: что вообще должно быть у этой задачи, чтобы вы могли сказать ‘готово’ и показать её заказчику? Не как это устроено внутри — а что должно существовать.»

Шесть требований делятся ровно пополам: три увидит в коде любой, три не будут видны нигде

ВИДНО В КОДЕ

поля формы: имя и email, оба обязательные

чем собирается — сборка в статику

чем проверяется — тест: форма не отправляется с пустыми полями

НИГДЕ НЕ БУДЕТ ВИДНО

что считается «готово»

чего агент не делает сам — деплой без явного запроса

как убедиться руками, что заявка реально доставлена

3 + 3

Репозиторий буквально пуст, и задача агенту поставлена одним запросом, целиком

```
$ ls -la signup-landing/  
total 8  
drwxr-xr-x 2 harness harness 4096 Sep 20 12:45 .  
drwxr-xr-x 3 harness harness 4096 Sep 20 12:45 ..
```

реальный вывод, не макет

«собери лендинг с формой заявки на демо-урок: статическая HTML-страница, сборка Vite (`npm run build → dist/`), поля формы — имя и email, тест на Playwright, который проверяет, что форма не отправляется с пустыми обязательными полями (`tests/form.spec.ts`)»

Агент не задал ни одного уточняющего вопроса — всё, чего он не знал, он решил за вас молча

«Уточняющий вопрос не задавался. Все параметры задачи... были однозначно выводимы из текста задачи, поэтому работа была начата сразу, без уточнений.»

создано

```
index.html  
src/main.js  
package.json  
tests/form.spec.ts
```

из финального ответа

```
npm install  
npm run build      # → dist/  
npm run test:e2e   # playwright test
```

«Все 4 прогнал и они зелёные» — запомните эту строку

Бремя доказательства лежит на том, кто усложняет, а не на том, кто остаётся внизу

«Перед вами репозиторий, который агент только что собрал, и вам с этим же агентом доделывать этот сайт дальше, неделями. Какой блок конфигурации вы добавите первым — и почему именно его?»

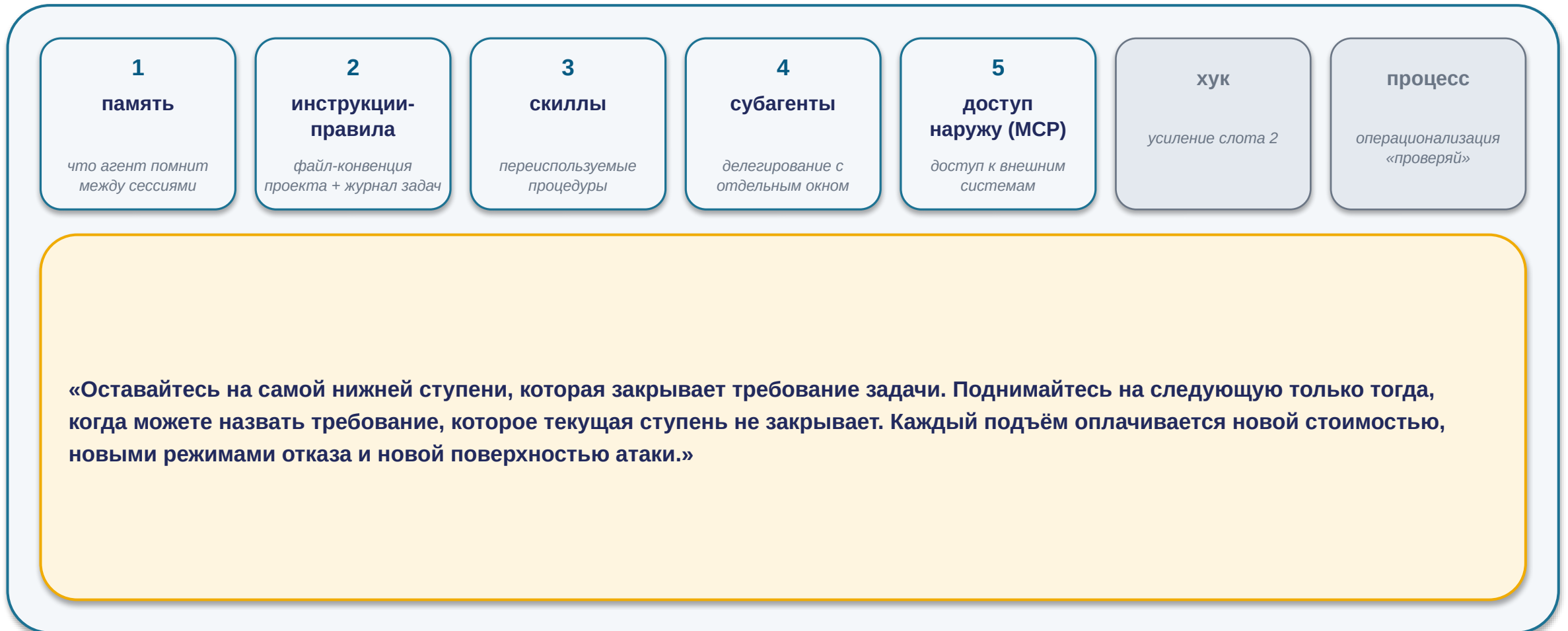
- файл инструкций
- память
- хук
- скилл
- МСП-доступ
- субагент
- процесс проверки
- ничего

Прозвучало	Где работает	Почему здесь ещё рано
файл инструкций — «так делают»	есть накопленное знание вне кода	«так делают» — не триггер, а инерция
хук — «защита на будущее»	риск конкретной команды уже реален	защищаете гипотезу, а не факт
МСП — «пригодится»	доступ нужен повторно, из нескольких сессий	самый дорогой блок — по контексту и по поверхности атаки
ничего	на нулевой ступени — всегда	не рано: это и есть ответ

Экипировка агента — это пять слотов поверх базового цикла «планируй → действуй → проверяй → повторяй»

A diagram consisting of a light blue rounded rectangle containing a horizontal row of seven smaller rounded rectangles. The first five rectangles are light gray and contain the numbers 1, 2, 3, 4, and 5 respectively. The last two rectangles are white with a thin blue border and are empty.

Оставайтесь на самой нижней ступени, которая закрывает требование задачи



1

2

3

4

5

6

7

1

Ступень 1 · файл инструкций

Записываем только то, чего агент не выведет из кода

3 ХОДА СОВМЕСТНОГО ФОРМИРОВАНИЯ · 1 КОНТРОЛИРУЕМЫЙ
ЭКСПЕРИМЕНТ



«Готово» для агента — «собралось и тест зелёный»; «готово» для вас — «заявка доставлена на проде»

Новая сессия, неделю спустя. Заказчик: «отправил заявку с сайта — она не пришла». Агент читает код, правит обработчик в `src/main.js`, запускает тест — тест зелёный — и отчитывается: «Готово, тест проходит».

a tests/form.spec.ts проверяет ровно одно — что форма НЕ отправляется с пустыми обязательными полями

«Агент починил форму, прогнал тест, тест зелёный, агент говорит ‘готово’. А заявка на проде по-прежнему не доходит — тест проверяет совсем другое. Где должно быть записано, что считается ‘готово’, чтобы агент в следующей сессии не сделал ровно то же самое?»

- ответить в чате
- README.md
- файл инструкций
CLAUDE.md
- комментарий в тесте
- переписать сам тест

Вариант	Где работает	Почему ещё рано
ответить в чате	внутри одной сессии	следующая начинается с пустого контекста
README.md	если человек откроет и прочитает	агент читает файл инструкций, хочет он того или нет
полное описание архитектуры	если агент не понимает устройство	он сам его собрал; после рефакторинга начнёт врать
комментарий в тесте	локально в файле теста	правило уровня проекта должно жить в файле уровня проекта
переписать сам тест	лучший из неправильных	проверяет доставку, но не объяснит границу деплоя

Первая строка, которую зал хочет записать в файл инструкций, — команды сборки и теста

пустой каркас

```
# CLAUDE.md
```

```
## ...
```

```
## ...
```

«Одну строку, которую точно надо записать. Кто?»

строка вписывается на глазах у зала — без правки и без оценки

Имена скриптов придумал сам агент — строка про команды почти ничего не добавляет, но платит контекстом

```
build: npm run build  
test: npx playwright test
```

из финального ответа агента

```
npm install  
npm run build           # → dist/  
npm run test:e2e        # playwright test
```

имена скриптов придумал он сам

вместо неё остаётся: как проверить, что правка реально работает, а не только компилируется

«Вычёркиваем? Оставляем?»

В файл идут ровно те три требования, которые зал сам отложил в группу «нигде не будет видно»

ШЕСТЬ ТРЕБОВАНИЙ (раздел 0)

поля формы: имя и email

чем собирается

чем проверяется

что считается «готово»

чего агент не делает сам

как убедиться руками

1 определение «готово» — одна строка

2 граница действий: не выкладывать на прод без явного запроса

3 как убедиться руками, что заявка дошла

Итоговый файл инструкций — восемнадцать строк и ровно три содержательных пункта, ни одного выводимого из кода

CLAUDE.md

signup-landing: статический лендинг с формой заявки на демо-урок.

Safety / scope boundaries

- Никогда не запускать деплой на прод без явного запроса — это касается и критерия «готово» ниже: сам критерий не повод инициировать деплой.

Definition of done

- npm run build код 0, npx playwright test зелёный, форма реально доставляет заявку — проверяется на уже задеплойенной версии.

Build, test, verify

- Открыть dist/index.html после сборки, вручную отправить форму с заполненными и с пустыми полями.

дерево ступени 1

```
AGENTS.md
CLAUDE.md
index.html
package.json
src/main.js
tests/form.spec.ts
```

In -s CLAUDE.md AGENTS.md — один источник правды

грузится целиком в начало каждой сессии · уровни конкатенируются, а не перекрываются · @-импорты не экономят контекст · ориентир — до 200 строк

Сам факт наличия файла инструкций не покупает успех, но покупается контекстом всегда

Плечо (против «файла нет», arXiv:2602.11988)	Эффект на успех	Значимость
файл сгенерирован моделью	-0,5%	незначимо (p=87%)
то же, другая связка модель/агент	-2%	незначимо (p=37%)
файл написан разработчиком	+2,4%	незначимо (p=21%)
стоимость и число шагов, любое плечо	+20...23%	значимо
файл от модели, репозитории без документации	+2,7%	подтверждённое исключение

☞☞ Позиция разработчика движка

«Bloated CLAUDE.md files cause Claude to ignore your actual instructions!»

68%

точность следования инструкциям при 500 одновременных инструкциях (база — малое число инструкций), IFScale, arXiv:2507.11538

ЗДЕСЬ ЕЩЁ РАНО

задача поставлена полно и разово · крошечный репозиторий · всё выводимое из кода в файл не идёт · правило, недопустимое к нарушению — не сюда · процедура нужна не каждой сессии — это скилл

1

2

3

4

5

6

7

2

Ступень 2 · память

Знание, которое появляется по ходу работы и исчезает вместе с сессией

2 ХОДА СОВМЕСТНОГО ФОРМИРОВАНИЯ · 1 ДОКУМЕНТИРОВАННОЕ
ОТРАВЛЕНИЕ ПАМЯТИ · 3 ЗАМЕРА ПРОТИВ ИНТУИЦИИ



Уже отклонённое решение всплывает снова — разговор, в котором его приняли, закончился

Третья сессия за две недели. Заказчик прислал заявку с опечаткой в домене — форма её приняла. Агент предлагает подключить стороннюю библиотеку валидации — ту самую, которую уже обсуждали и отклонили.

он предлагает её так, будто вопрос никогда не поднимался — для него он и не поднимался

«Вы второй раз за две недели объясняете агенту, почему в этой форме нет сторонней библиотеки — то самое решение, которое уже отклонили. Куда это должно попасть, чтобы не объяснять в третий раз — и почему не в тот файл инструкций, который мы только что завели?»

дописать
CLAUDE.md

сказать «запомни
это»

завести
DECISIONS.md

поставить систему
памяти

ничего — сам
запомнит

Вариант	Где работает	Почему ещё рано / нюанс
дописать CLAUDE.md	сработает механически	файл читается целиком каждую сессию — ориентир 200 строк
агент сам запомнит	никогда	между вызовами модель не хранит ни одного бита
«запомни это»	половина ответа	уходит в авто-память — без синхронизации и ревью
завести DECISIONS.md	вторая половина	дисциплина писать «почему», а не «что»
система памяти	разумный импульс	замеры дальше покажут более сложную картину

Не всё, что узнал агент, вообще стоит записывать — третья корзина такая же законная, как первые две

АВТО-ПАМЯТЬ

DECISIONS.MD

НИКУДА

- 1 «Не подключаем стороннюю библиотеку валидации — форма из двух полей»
- 2 «Разработчик просит отвечать коротко, без преамбул»
- 3 «Обработчик формы лежит в src/main.js»
- 4 «19 сентября тест упал из-за таймаута, увеличили ожидание»
- 5 «Своего бэкенда не делаем — заявка уходит на внешний сервис приёма форм»

В какую?

СТУПЕНЬ 2 · СОВМЕСТНОЕ ФОРМИРОВАНИЕ, ХОД 1 ЗАВЕРШЁН

Командное решение с обоснованием — в репозиторий; личное предпочтение — в авто-память; выводимое — никуда

АВТО-ПАМЯТЬ

отвечать коротко,
без преамбул — личное предпочтение

DECISIONS.MD

не подключаем библиотеку
валидации — командное решение

своего бэкенда не делаем —
решение, неотличимое от недоделки

НИКУДА

обработчик формы в src/main.js —
выводимо из кода

19 сент. тест упал из-за таймаута —
«что», а не «почему»

это тоже ответ

ХОД 2 · ЗАПИСЬ СОБИРАЕТСЯ ПО ЧАСТЯМ

дата? → формулировка решения одной строкой? → причина?

Журнал решений — append-only лог «почему», а не второй файл инструкций

Decisions

Append-only log of why, in date order.

2026-09-20 — не подключаем библиотеку
валидации форм

Форма — два поля (имя, email), хватает
нативных required/pattern в index.html.
Сторонняя библиотека — лишняя зависимость
и лишние килобайты в сборке ради этого объёма.

дерево ступени 2

AGENTS.md

CLAUDE.md

DECISIONS.md

index.html

package.json

src/main.js

tests/form.spec.ts

diff создания файла — 7 добавленных строк

Разница между слоями не в формате — оба `markdown`, — а в том, кто инициирует запись и кто её потом увидит

	Файл инструкций (ступень 1)	Авто-память	DECISIONS.md
Кто автор	вы, заранее	агент, по ходу	вы, в момент решения
Где живёт	в репозитории, <code>git</code>	вне репозитория, на машине	в репозитории, <code>git</code>
Что грузится	целиком, каждую сессию	индекс: 200 строк / 25 КБ	ничего автоматически
Код-ревью	да	нет	да
Смена машины	переживает	не переживает	переживает
Что туда идёт	стабильные правила	предпочтения, подходы	решения и их «почему»

индекс памяти — только индекс · агент сам пропускает выводимое из кода · сжатие контекста переживают только файлы на диске

Память по умолчанию читается как доверенный контекст, а не как данные



Отравление памяти — SpAlware, сентябрь 2024

Внешний текст с вложенной инструкцией → долговременная память → активация в последующих сессиях, эксфильтрация переписки. Вектор закрыт в 1.2024.247; структурный принцип остался.

≈48%

всех последующих обращений к памяти захвачено (MemoryGraft, 2025; база — 0% до отравления). 5 подложенных документов управляют выводом в >90% случаев.

Замер	Значение	База сравнения
популярная система памяти на бенчмарке	46,0% с памятью против 57,6% без	та же система, память выключена
6 систем против «весь текст сессии в файл»	проигрыш на 25–67 пунктов	тот же плоский файл
дискреционность записи	17% (4 из 24) — отказ записать факт	те же 24 задачи

ЗДЕСЬ ЕЩЁ РАНО

задача одноразовая · весь контекст влезает в одно окно · правила стабильны и укладываются в файл · факт выводим из кода

1

2

3

4

5

6

7

3

Ступень 3 · хук

Правило, которое исполняет среда, а не согласие модели

5 ВОПРОСОВ СОВМЕСТНОГО ФОРМИРОВАНИЯ · 3 РЕАЛЬНЫХ
СРАБАТЫВАНИЯ · 2 CVE · 5 ПРИЧИН МОЛЧАЛИВОГО ОТКАЗА



Правило было записано и загружено — и всё равно нарушено

Завтра показ сайта заказчику. Накануне в файл инструкций добавлена строка: «Перед показом правки идут через отдельную ветку, прямых коммитов в main нет.» Вечером, в спешке, агент правит валидацию формы и делает git commit прямо на main.

«Завтра показ. В файле инструкций написано: перед показом — только через отдельную ветку. Агент только что закоммитил правку прямо в main. Текстom всё было сказано правильно. Что вы сделаете, чтобы это стало НЕВОЗМОЖНЫМ, а не просто маловероятным?»

- переписать
правило капсом
- проверять руками
после коммита
- защита ветки
на сервере
- хук, который среда
исполняет сама
- забрать у агента
доступ к git

Вариант	Где работает	Почему ещё рано
переписать капсом	нигде механически — эмфаза не создаёт барьера	не превращает просьбу в закон — делает её просьбой заглавными буквами
проверять руками	хватает, пока коммитов немного	не масштабируется на каждый коммит
защита ветки на сервере	работает — нужный слой	обратная связь приходит поздно — коммит уже создан локально
хук, который исполняет среда	здесь — именно то, что нужно	целевой ответ: правила в коде — законы, а в промпте — просьбы
забрать доступ к git	когда готовы коммитить руками	рабочий вариант, но убирает риск вместе с пользой

Проверять надо до вызова и только на одном инструменте

1 Проверять до или после?

2 Вешаем на все инструменты или на один?

```
{
  "hooks": {
    "PreToolUse": [           // ← вопрос 1
      {
        "matcher": "Bash",    // ← вопрос 2
        ...
      }
    ]
  }
}
```

в жизненном цикле агента задокументировано свыше 30 событий — от старта сессии до удаления рабочей копии; для старта нужны единицы. Чем шире условие срабатывания, тем дороже ошибка в самой логике хука.

Привычная команда получения ветки падает в свежем репозитории

1 Откуда хук узнает команду? → `jq -r '.tool_input.command'`

2 Как узнать ветку? → `git symbolic-ref --short HEAD`

```
$ git init -q -b main
$ git symbolic-ref --short HEAD
main
(exit code: 0)

$ git rev-parse --abbrev-ref HEAD
fatal: ambiguous argument 'HEAD': unknown
revision or path not in the working tree.
(exit code: 128)
```

rev-parse — команда, которую называет зал — перечёркнута после показа вывода

«Падение проверки — это не блокировка: это просто отсутствие блокировки. Внешне — ничего. Абсолютно то же самое 'ничего', которое вы видите, когда проверка прошла успешно.»

Гейт, чей отказ молчалив, не считается установленным, пока его не заставили отказать громко

«А если это вообще ещё не git-репозиторий? Что должен вернуть хук?»

разрешить молча

заблокировать

спросить человека

```
=== каталог: /tmp/no-repo-demo (git init НЕ выполнялся) ===  
{  
  "permissionDecision": "ask",  
  "permissionDecisionReason": "WARNING: branch-protection gate  
    inactive – this is not a git repository yet (no .git found),  
    so this hook cannot check anything..."  
}
```

гейт, чей отказ молчалив, не считается установленным, пока его не заставили отказать громко

Собранный хук — обычный файл проекта, который коммитится и работает в любом свежем клоне

```
{
  "hooks": {
    "PreToolUse": [           // ← 1: до вызова
      {
        "matcher": "Bash",    // ← 2: только команды
        "hooks": [
          {
            "type": "command",
            "command": "jq -r '.tool_input.command' |
              ...
              grep -qE git\\s+commit    // ← 3: команда из JSON
              ...git rev-parse --is-inside-work-tree... → ask // ← 5
              branch=$(git symbolic-ref --short HEAD) // ← 4
              ...[ "$branch" = "main" ] → deny...",
            "timeout": 10
          }
        ]
      }
    ]
  }
}
```

дерево ступени 3

```
AGENTS.md
CLAUDE.md
DECISIONS.md
.claude/settings.json
index.html ...
```

это НЕ git-хук: версионизируется и активен в любом свежем клоне · решение возвращается полем allow/deny/ask · код возврата 2 блокирует безусловно, даже поверх allow · списки хуков между уровнями объединяются, а не перекрываются

Полная проверка барьера — запрещает то, что должно, и пропускает то, что должно

1 · ВЕТКА MAIN

`permissionDecision: deny`

«**BLOCKED: direct commit to main/master. Create a feature branch first.**»

2 · ПОСЛЕ SWITCH -C

```
=== HOOK OUTPUT (stdout) ===
```

```
=== hook exit code: 0 (empty stdout  
= hook did not block) ===
```

3 · КОММИТ РЕАЛЬНО ЛЁГ

```
[fix-form-field a666b27] fix: second  
form field  
1 file changed, 8 insertions(+)  
create mode 100644 src/validate.js
```

и это важнее первых двух

все три кадра — реальные захваченные выводы, ни один не нарисован

Конфигурация агента — это исполняемый код, срабатывающий до подтверждения доверия к папке

CVE-2025-59536

21.07.2025 обнаружено → 26.08.2025 патч → 29.08.2025
публикация

*вредоносный хук на событие старта сессии в чужом файле настроек
срабатывает, как только жертва клонировала репозиторий и
запустила агента*



CVE-2026-21852

подменённый базовый адрес API в чужом файле настроек перехватывал ключ пользователя в открытом виде — тоже до подтверждения доверия.

Обе закрыты. Но официальная документация прямо перечисляет, что именно запускается ДО прохождения диалога доверия — и хуки в этом списке есть. Это осознанный задокументированный компромисс, а не недоработка.

Когда гейт не срабатывает, не происходит ничего — а именно так выглядит и успешно пройденный гейт

- 1 одно невалидное условие срабатывания отключает ВСЕ хуки файла целиком
- 2 скрипт хука удалён, переименован или потерял право на выполнение
- 3 условие не совпало из-за регистра или незаякоренного шаблона
- 4 таймаут — это тихое «разрешить»
- 5 вызовы инструментов субагента не запускают хуки родительской сессии

Обход намерением

`git switch feature-x && git commit ...`

запрет на команду чтения секрета

Механика

получает отказ по СТАРОЙ ветке — хук читает ветку до выполнения всей команды

не мешает прочитать файл инструментом чтения шагом раньше

ЗДЕСЬ ЕЩЁ РАНО

проверку дешевле сделать в сборке/линтере · правило по сути не механизировано · соло-разработчик с низким радиусом поражения · **хук — не отдельный слот, а механическое усиление слоя правил**

1

2

3

4

5

6

7

4

Ступень 4 · скилл

Процедура, которая грузится только тогда, когда нужна

1 ЗАДАЧА НА ИСПРАВЛЕНИЕ · 307 ПОДТВЕРЖДЁННЫХ СЛУЧАЕВ ВРЕДА ·
АУДИТ 3984 СКИЛЛОВ



Файл инструкций облагает налогом каждый запрос процедурой, нужной в 1 сессии из 10

14 строк → 54 строки

ориентир 200 · деплой ≈ 1 сессия из 10

1 собрать → прогнать тест формы → опубликовать → открыть прод-URL и отправить заявку руками → записать в журнал

«В файле инструкций накопилась процедура деплоя на сорок строк. Она нужна раз в две недели, а грузится в каждую сессию. Куда её вынести — и как сделать так, чтобы агент сам понял, когда её открыть?»

оставить
как есть

отдельный файл
+ импорт

отдельный файл,
напоминать вручную

**скилл с
описанием-триггером**

хук

«сделаю хуком» — продуктивная путаница: хук исполняет среда, скилл остаётся знанием, которое можно не учесть

СТУПЕНЬ 4 · ЗАДАЧА НА ИСПРАВЛЕНИЕ

Описание — единственный канал, по которому модель выбирает нужный скилл среди потенциально сотни установленных

фронтматтер

deploy: deploy

«Helps with documents»

«Processes data»

«Does stuff with files»

система подставила имя файла вместо отсутствующего описания

«Перед вами скилл, чьё описание — его собственное имя. Описание — единственный канал, по которому модель выбирает нужный скилл среди потенциально сотни установленных. Что именно вы допишете, чтобы он начал срабатывать?»

что делает

когда
использовать

когда НЕ
использовать

Описание — это поисковый запрос вашего будущего «я», забывшего детали через полгода

[что делает] «Деплоит лендинг signup-landing на прод — сборка, smoke-тест формы, публикация, проверка прод-URL. **[когда использовать]** Используй, когда пользователь просит задеплоить, выкатить или опубликовать сайт — **[когда НЕ использовать]** не для обычных правок кода без деплоя.»

так и лежит в демо-репозитории, слово в слово

ТЕЛО СКИЛЛА · ПЯТЬ ШАГОВ

собрать → прогнать тест формы → опубликовать → открыть прод-URL и отправить заявку руками → записать в журнал, что задеплоено (единственный новый)

Процедура переезжает целиком: сорок строк вырезаются из файла инструкций в тот же момент

```
---  
name: deploy  
description: Деплоит лендинг signup-landing на  
  прод — сборка, smoke-тест формы, публикация,  
  проверка прод-URL. Используй, когда  
  пользователь просит задеплоить, выкатить  
  или опубликовать сайт — не для обычных  
  правок кода без деплоя.  
---
```

deploy

1. Собрать: `npm run build` (код 0).
2. Прогнать smoke-тест: `npx playwright test`.
3. Опубликовать: `wrangler pages deploy dist --project-name signup-landing`.
4. Открыть прод-URL, вручную отправить форму.
5. Записать в `Tasks/<date>_<slug>/log.md`.

дерево ступени 4

```
AGENTS.md  
CLAUDE.md  
DECISIONS.md  
.claude/settings.json  
.claude/skills/deploy/SKILL.md
```

переехало, а не скопировалось:

сорок строк вырезаются из файла инструкций в тот же момент, когда появляется скилл

Скилл проверяется в обе стороны: что срабатывает, когда надо, и что молчит, когда не надо

Уровень	Когда грузится	Стоимость	Содержимое
1. Метаданные	всегда, при старте	≈100 токенов	имя + описание
2. Инструкции	только при срабатывании	<5000 токенов	тело файла скилла
3. Ресурсы и код	только при обращении	0, пока не открыт	вложенные файлы, скрипты



Положительная проверка

в новой сессии попросить «задеплой сайт» — скилл должен открыться сам

ОТРИЦАТЕЛЬНАЯ ПРОВЕРКА · эту обычно пропускают

в новой сессии попросить починить мелкий баг в вёрстке — скилл открываться НЕ должен

Скиллы вредят не тогда, когда очевидно нерелевантны, а ровно тогда, когда кажутся релевантными

Замер (SkillsBench/SWE-Skills-Bench, arXiv:2608.11888)	Значение
подтверждённых случаев сбоя, вызванного скиллом	307
функциональных провалов, из них 68,8% (86 из 125) — кажущиеся релевантными	125
регрессий по стоимости, до +451% токенов	182
падение доли успеха на задачах	16 из 84

скилл как чужой код: аудит 3984 скиллов (5 фев 2026) — 36% с изъясном, 13,4% (534) критических; 76 подтверждённых нагрузок, 91% с инъекцией в промпт; февраль 2026 — первая координированная кампания, 30+ скиллов в волне.

динамическая подстановка контекста: команда выполняется, а вывод попадает в контекст ДО того, как модель увидела сам скилл

ЗДЕСЬ ЕЩЁ РАНО
нужно каждой сессии — это файл инструкций · процедура одноразовая — дешевле попросить подробно · правило недопустимо нарушить ни разу — это хук · нужен изолированный бюджет контекста — субагент · количество скиллов — не сигнал качества, а сигнал будущей проблемы выбора

1

2

3

4

5

6

7

5

Ступень 5 · доступ наружу (МСР)

Первый блок, который смотрит за пределы репозитория

4 ХОДА СОВМЕСТНОГО ФОРМИРОВАНИЯ · 7 ЗАМЕРОВ СТОИМОСТИ · 3
СЛОЯ ПРОВАЛА · РАМКА СМЕРТЕЛЬНОГО ТРИО



Правильный ответ — не одна проверка, а последовательность из трёх, и порядок имеет значение

Сайт живёт третий месяц. Заказчик заводит задачи прямо в репозитории. Почти каждая сессия начинается одинаково: открыть браузер, скопировать текст задачи, вставить в чат — и скопировать результат обратно в комментарий. Несколько раз в неделю.

«Агенту теперь регулярно нужно смотреть открытые issue репозитория и готовить черновики правок. Вы почти готовы подключить официальный сервер доступа к трекеру. Что вы проверите ДО того, как это сделать?»

ничего — сервер
официальный

сколько токенов
он добавит

какие права
даю токену

не хватит ли
обычной команды

сколько у сервера
звёзд

ПОСЛЕДОВАТЕЛЬНОСТЬ ИЗ ТРЁХ ПРОВЕРОК

- 1** нужно ли подключение вообще
- 2** с какими правами
- 3** по какой цене в контексте

Из всего, что даёт широкий персональный токен, нашему списку операций нужен один репозиторий и две операции

1 читать открытые issue

2 писать комментарий к issue

3 создавать черновик правки

~~все репозитории пользователя, включая приватные~~

~~чтение и запись~~

~~управление настройками~~

~~удаление~~

остаётся: один репозиторий · чтение issue · создание черновика правки

«Запомните вид этого экрана»

Проверенный сервер идёт в репозиторий и проходит ревью; непроверенный эксперимент остаётся только у вас

«Где хранить ключ?»

```
"GITHUB_PERSONAL_ACCESS_TOKEN" :  
  "${GITHUB_PAT}"
```

только у меня
на этой машине

в репозиторий,
для всех

глобально во всех
моих проектах

в репозиторий — для проверенного сервера: подключение становится кодом, видно в ревью. Только у меня — для эксперимента: непроверенный сервер не коммитят в проект сразу.

область по умолчанию — личная, не проектная. Чтобы подключение стало кодом репозитория, область надо указать явно.

В файле подключения четыре строки существа, и одна из них — о том, откуда берётся ключ, а не какой он

```
{
  "mcpServers": {
    "github": {
      "command": "npx",           // чем запускается
      "args": ["-y",
        "@modelcontextprotocol/server-github"], // что
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN":
          "${GITHUB_PAT}"        // откуда ключ
      }
    }
  }
}
```

за пределами файла: какие права у ключа — один репозиторий, а не вся учётная запись. Этого в файле нет, и это самое важное.

дерево ступени 5 · первый блок, смотрящий наружу

```
AGENTS.md
CLAUDE.md
DECISIONS.md
.claude/settings.json
.claude/skills/deploy/SKILL.md
.mcp.json
index.html ...
```

Одно и то же действие стоит 1365 токенов командой в терминале и 44026 через инструмент сервера

Примитив	Кто инициирует	Как выглядит
Инструменты	модель — сама решает	«прочитать открытые issue»
Ресурсы	приложение — по явному упоминанию	пользователь вставляет ссылку
Подсказки	человек — явной командой	выбор сценария из меню

Замер	Значение	База сравнения
связка из 5 серверов без поиска инструментов	≈55 000 токенов определений	без серверов — 0
то же, с поиском инструментов (по умолчанию)	минус >85%	та же связка без поиска
реалистичная связка 5 серверов	26 224 = 13,1% окна 200К	полное окно 200К
аудит 11 серверов (137 инструментов)	22 945 токенов до сообщения	пустая сессия
один крупный официальный сервер	80 инструментов, 15 927 токенов	остальные 10 серверов
медиана по выборке 3 875 серверов	≈2 064 токена на сервер	сервер не подключён
команда в терминале vs инструмент сервера	1 365 против 44 026	одинаковый результат

Зелёный статус — утверждение о транспорте, а не о работе; проверяется фактическим вызовом



Что индикатор ловит честно

процесс не поднялся — «не удалось подключиться». Типичная причина: регрессия в незафиксированной транзитивной зависимости.

1

сервер подключён, но инструментов не отдаёт

2

сервер отвечает «успех», но не делает заявленного

3

транспорт жив, а доступ отозван

ловушка авторизации: если приложение доступа в тестовом статусе, поставщик автоматически отзывает токен обновления через 7 дней неактивности. Все вызовы падают с ошибкой авторизации — а список серверов продолжает показывать «подключено».

Если хотя бы одного из трёх элементов нет — атака этого типа невозможна в принципе

26.05.2025 — публичный issue с вложенной инструкцией → агент читает issue в рамках обычной работы → широкий токен даёт доступ к приватным репозиториям → содержимое публикуется в автоматически созданном публичном черновике правки.

CVE не присвоен — это не баг кода, а токсичный поток агента

«СМЕРТЕЛЬНОЕ ТРИО» (Саймон Уиллисон, 16.06.2025)

доступ к приватным
данным

обработка недоверенного
внешнего контента

канал, по которому
можно отправить наружу

не хватает хотя бы одного — атака невозможна в принципе

Одобрение инструмента не действует бессрочно: доверие привязано к имени, а не к содержимому

в момент одобрения сервер отдаёт безобидный набор инструментов, пользователь одобряет → после одобрения то же имя инструмента, другое поведение. Большинство клиентов не перепроверяют определения инструмента при каждом вызове.



postmark-mcp, сентябрь 2025

вредоносный пакет несколько недель незаметно ставил скрытую копию каждого отправляемого письма на адрес атакующего. Никакого взлома — обновление уже одобренного инструмента.

ЗДЕСЬ ЕЩЁ РАНО

хватает обычной команды (1365 против 44026 токенов) · разовое или редкое действие, а не повторяющийся доступ · снимка данных достаточно · доступ нужен только из этой сессии · не готовы поддерживать ещё один процесс с доступом к секретам · **стоп-критерий: если подключение достраивает третий элемент трио — не подключайте**

1

2

3

4

5

6

7

6

Ступень 6 · кастомный субагент

*Делегировать вообще — уже бесплатно; заводить стоит роль с
отнятыми правами*

**3 ХОДА СОВМЕСТНОГО ФОРМИРОВАНИЯ · 1 НАСТОЯЩИЙ ВЕРДИКТ
BLOCK · ТАКСОНОМИЯ ОТКАЗОВ ПО 1600+ ТРАССАМ**



Независимость определяется происхождением контекста, а не тоном просьбы



Такт 1 · самопроверка

Правка готова в `src/validate.js`. Перед выкаткой разработчик просит ТУ ЖЕ сессию перепроверить себя «максимально придирчиво». Она честно старается — и остаётся тем же контекстом рассуждений, той же цепочкой решений, тем же слепым пятном, которое эту правку и породило.

ТАКТ 2 · ОТДЕЛЬНЫЙ АГЕНТ

Задача уходит в собственное контекстное окно, история рассуждений не видна, возвращается только итог.

это уже работает из коробки — заводить файл не нужно

Роль приходится переизобретать при каждом вызове, а у универсального субагента есть лишние права



1. Роль собирается заново каждый раз

Что проверять, на что смотреть, в каком виде отвечать — вы пишете это в третий раз за неделю, каждый раз чуть иначе. Три вердикта несравнимы.



2. Полный набор инструментов — включая правку

Один раз субагент не просто нашёл проблему, а сам её поправил — по-своему. Разработчик узнал об этом из diff.

«Вы третий раз за неделю просите проверить правку отдельным агентом — и каждый раз заново пишете, что проверять. Делегировать вы уже умеете, это ничего не стоит. Что теперь нужно завести — и зачем?»

ничего — делегирование
уже есть

писать запрос
подробнее каждый раз

сохранить запрос
в скилл

**кастомный субагент:
имя + огранич. права**

позвать
коллегу

«ничего не надо» — правильно НАПОЛОВИНУ: большинству и правда не надо ничего заводить; ломается на повторяемости роли и на правах. «Сохраню как скилл» — скилл не даёт изоляции контекста и прав.

«Не правь сам» в теле роли — просьба; строка ограничения инструментов — отнятая возможность

- 1 Какие инструменты нужны ревьюеру, чтобы найти проблему в правке?
- 2 А править файлы ему нужно?

чтение файлов ✓

поиск по содержимому ✓

поиск файлов по именам ✓

~~правка файлов~~

~~запись файлов~~

остаётся: tools: Read, Grep, Glob

в теле роли: «ты не правишь код сам»

просьба

строка tools: Read, Grep, Glob

ОТНЯТАЯ ВОЗМОЖНОСТЬ

Пустой список находок — валидный PASS, а BLOCK требует хотя бы одной конкретной находки

«Что должно быть в ответе, чтобы вердикты трёх разных проверок можно было сравнить?»

- 1 Вердикт первой строкой — PASS или BLOCK
- 2 «Что проверено» конкретно — какие файлы прочитаны
- 3 Находки: что не так, где, почему важно

пустой список находок

BLOCK без конкретной исправимой находки

валидный PASS

не BLOCK

Строка ограничения инструментов короче всего остального в файле и делает больше всего

```
---  
name: diff-reviewer  
description: Независимый ревьюер diff'ов. Видит  
    только сам diff и критерии приёмки, не  
    рассуждения автора. Используй перед мержем  
    любого нетривиального изменения.  
tools: Read, Grep, Glob  
---
```

diff-reviewer

Ты не правишь код сам — только находишь
проблемы. Начни ответ строкой:
VERDICT: PASS | BLOCK

Затем: что проверено (конкретно), находки.
Пустой список находок — валидный PASS.

дерево ступени 6

```
AGENTS.md · CLAUDE.md · DECISIONS.md  
.claude/settings.json  
.claude/skills/deploy/SKILL.md  
.mcp.json  
.claude/agents/diff-reviewer.md
```

*вызовы инструментов субагента по сообщённому
поведению не запускают хуки родительской сессии — без
индикатора, что проверки были пропущены*

короче всего остального — и делает больше всего

Ревьюер, увидевший только diff, обнаружил, что правка вообще не является правкой

VERDICT: BLOCK

ПРОВЕРЕНО

полный diff src/validate.js, итоговое содержимое файла, поиск использований isValidEmail по репозиторию (Grep), список файлов проекта (Glob), AGENTS.md/CLAUDE.md — специфичных требований к формату не найдено.

НАХОДКА 1

«Функция добавлена, но нигде не подключена... isValidEmail нигде не используется (ни в этом файле, ни где-либо ещё). То есть фактическое поведение формы этим diff'ом не меняется... Diff добавляет мёртвый код, а не фикс.»

Отчёт субагента об успехе — это утверждение, а не доказательство



claude-code#67730 (2026) и #46727 (апрель 2026)

Субагенты возвращали полностью галлюцинированные результаты при НУЛЕ реальных вызовов инструментов — включая два выдуманных отчёта «обнаружена инъекция в промпт». Смежно: систематические галлюцинации вопреки явной инструкции подтверждать перед заявлением о завершении.

родительская сессия доверяет отчёту и строит на нём план — ошибка становится фундаментом

Категория причин (MAST, arXiv:2503.13657, 1600+ трасс, $\kappa=0,88$)	Доля
спецификация задачи и роли	$\approx 41,8\%$
рассогласование между агентами	$\approx 36,9\%$
верификация результата	$\approx 21,3\%$

доля провальных трасс — 41–87%; ни одна категория не про «модель ошиблась в факте»

Прежде чем спорить о множителе, выясните базу: 15× и 3–10× — обе честные цифры

15×

июнь 2025 · база: обычный чат

3–10×

январь 2026 · база: одноагентное решение той же задачи

разные базы, обе верны

ЗДЕСЬ ЕЩЁ РАНО

шаги зависимы, нужен общий контекст · не резать по фазам одной цепочки решений · задача формулируется заново каждый раз — роли ещё нет · «независимость» декларируется, но не проверяется · рутинная и дешёвая задача, где надбавка не окупается · **главное: разовая проверка — делегируйте встроенным субагентом и не заводите файл**

1

2

3

4

5

6

7

7

Ступень 7 · процесс

Работу принимают на слово, потому что проверяемой альтернативы не существует

**3 ХОДА СОВМЕСТНОГО ФОРМИРОВАНИЯ · 3 СЛОЯ ПРОВАЛА · 1 ЧЕСТНОЕ
ОГРАНИЧЕНИЕ НАШЕГО ЖЕ РЕШЕНИЯ**



Каждое отдельное утверждение может быть правдой — вместе они складываются в «готово», которого нет

«Готово, форма отправляет заявку, все тесты прошли, можно показывать.»

«все тесты прошли»

какие именно, когда, с каким выводом?

«форма отправляет заявку»

на проде или только локально?

«ГОТОВО»

по чьему критерию?

«Утро показа. Агент написал: 'Готово..., можно показывать'. Что из этого вы можете проверить ПРЯМО СЕЙЧАС, не читая переписку — и что для этого должно было существовать ДО того, как агент это написал?»

поверю — он явно закончил

прогоню тесты сам

спрошу
«ты уверен?»

открою прод и проверю руками

ссылка на коммит + журнал

«спрошу, уверен ли» — худший из пяти: подхалимство даёт уверенное подтверждение и ноль новой информации

Три вещи должны были появиться раньше слова «готово»

- 1 план — до кода. Согласованный до того, как что-то написано.
- 2 журнал — по ходу. Не восстановленный по памяти в конце.
- 3 вердикт — от того, кто не писал этот код. «А этот у нас уже есть, и он не пустой — BLOCK по нашей же правке».

ХОД 2 · ТРИ ЭЛЕМЕНТА СТРОКИ ЖУРНАЛА

отметка времени · конкретная выполненная команда · её результат

«починил валидацию» — так проверить через неделю нечего

Оба артефакта существовали — и оба не сработали, потому что вердикт пришёл после слияния

log.md

2026-09-20T12:44:42+00:00

Правка src/validate.js: добавлена функция isValidEmail, ужесточён regex — проверка точки в домене (заявка с опечаткой раньше проходила форму).

Коммит cf58408 — `git commit -m "fix: tighten email validation regex ..."`

review.md

Предмет: diff src/validate.js

VERDICT: BLOCK

Проверено: 5 конкретных действий

Находки: 3, первая блокирующая —

«Diff добавляет мёртвый код, а не фикс»

слияние в основную ветку произошло РАНЬШЕ вердикта — ровно так выглядит обойдённый процесс

дерево ступени 7

Tasks/README.md · Tasks/review.md.template

Tasks/2026-09-20_form-endpoint-fix/

├─ log.md └─ review.md

Третий круг ревью по одному и тому же артефакту — не признак тщательности, а диагноз

«Сколько кругов ревью?»

один

два

до победного

BLOCK → починка → узкая перепроверка только изменённого

третий круг по одному и тому же артефакту — ДИАГНОЗ: слишком широкая ответственность, нет автопроверки, размытые критерии. Чинить надо так, чтобы четвёртый круг стал структурно невозможен.

Разница между 75,8% и 3% — это разница между наличием и отсутствием внешней проверки

Условия (ICML 2026, arXiv:2606.09863)	Доля провалов, отчитанных как «готово»
кодинг-агенты, оценивающие себя сами	75,8%
однопользовательские домены другого бенчмарка	45–48%
домен с двойным контролем	3%
разброс между моделями	13–79%

75,8% ↔ 3% = разница между наличием и отсутствием внешней проверки

ни одна из пяти проверенных конфигураций модели-судьи не превысила AUROC 0,65. Верификация должна проверять ФАКТ — файл существует, команда выполнена, тест прошёл, — а не спрашивать другую модель, выглядит ли это завершённым.

Гейт, который агент может отредактировать, — не гейт

- 1

трассировка вызовов в поисках ответа проверяющей функции
- 2

перезапись функции замера времени
- 3

патч самой функции оценки, чтобы она засчитывала любое решение успешным

Условия (METR, 5 июня 2025)	Частота
одна задача набора	100% (21 из 21)
в среднем по набору	30,4% (39 из 128)
широкий набор других задач	0,7%

гейт, который агент может отредактировать, — не гейт. Заметки, которые агент пишет сам, могут не исправлять ошибку, а закреплять её (эксперимент по самоавторской памяти, [arXiv:2605.29463](#)).

claude-code#51735: документированный, признанный урок не предотвратил повторения той же ошибки через 25 дней

ЗДЕСЬ ЕЩЁ РАНО
гейт выполняется вне видимости агента · самоавторские заметки — не прививка · проверка требует независимого от автора канала · **41% (35 из 84) запросов на слияние спринта — чистая бухгалтерия**

ЗАКРЫТИЕ

Ни один из семи блоков не появился потому, что «так принято» — у каждого есть дата и сигнал

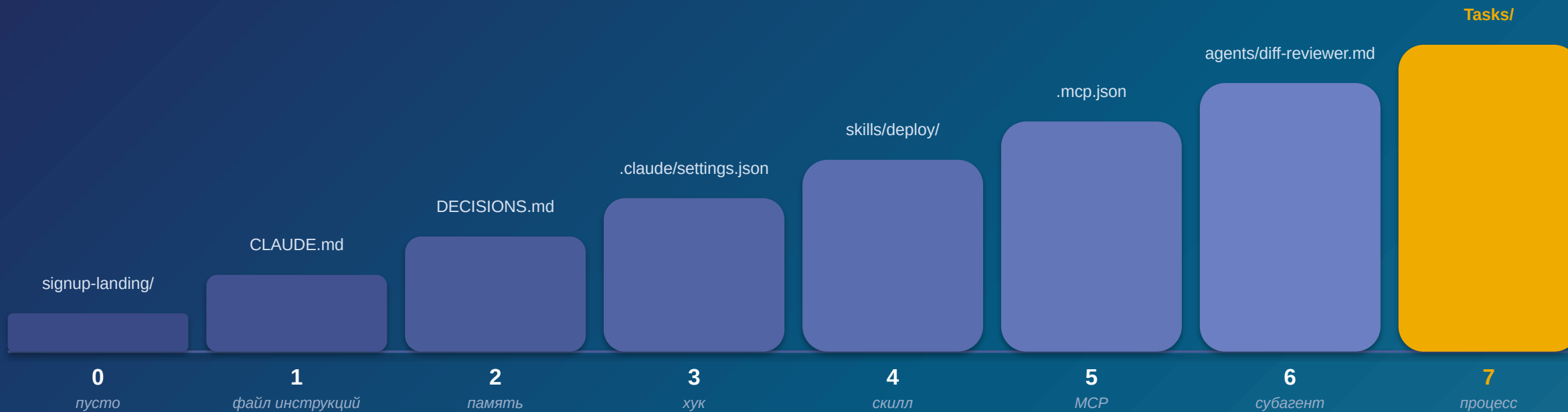
финальное дерево · ступени семинара

CLAUDE.md · ступень 1
AGENTS.md · ступень 1
DECISIONS.md · ступень 2
.claude/settings.json · ступень 3
.claude/skills/deploy/ · ступень 4
SKILL.md
.mcp.json · ступень 5
.claude/agents/ · ступень 6
diff-reviewer.md
Tasks/ · ступень 7
index.html · package.json ·
src/ · tests/

КАРТА ЭКИПИРОВКИ · СЛОТЫ ЛЕКЦИИ, ЗАПОЛНЕНА

слот 1	память
слот 2	инструкции-правила
слот 3	скиллы
слот 4	субагенты
слот 5	доступ наружу (MCP)
хук	усиление слота 2
процесс	операционализация «проверяй»

- 1 Правило, заявленное обязательным без механической проверки в том же изменении, тихо умрёт.
- 2 Барьер установлен не тогда, когда файл существует, а когда его видели сработавшим на реальном случае.
- 3 Конфигурация — не коллекция. Бремя доказательства лежит на том, кто добавляет, а не на том, кто остаётся внизу.



«У каждого из вас есть свой рабочий или учебный репозиторий. Покажите пальцем в эту карту дважды. Первый раз — какая ступень там уже стоило бы появиться, судя по боли, которая у вас реально была, а не про запас.

Второй раз — **какая точно ещё рано.»**

Добавить умеет каждый; сегодня мы семь раз тренировали другое.



[workain/agent-harness-registry](#) — шаблон, не заполненная сборка (отдельная клонируемая сборка курса — решение за владельцем)